COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	July 17, 2026	**DATE SUBMITTED:**	July 17, 2026
NAME:	Clifford Lee C. Doplayna	**INSTRUCTOR:**	Engr. Jimlord Quejado

2.1 Hands-on Activity 2.1 Arrays, Pointers and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

Table 2-1. Initial Driver Program

Screenshot

```cpp
#include <iostream>
#include <string.h>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    //constructor
    Student(std::string newName = "John Doe", int newAge = 18) {
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    }

//deconstructor
~Student(){
std::cout << "Destructor Called." << std::endl;
}
//Copy Constructor
Student(const Student &copyStudent){
std::cout << "Copy Constructor Called" << std::endl;
studentName = copyStudent.studentName;
studentAge = copyStudent.studentAge;
}
//Display Attributes
void printDetails(){
std::cout << this->studentName << " " << this->studentAge << std::endl;
}
};

int main() {
Student student1("Roman", 28);
Student student2(student1);
Student student3;
student3 = student2;
return 0;
}
```

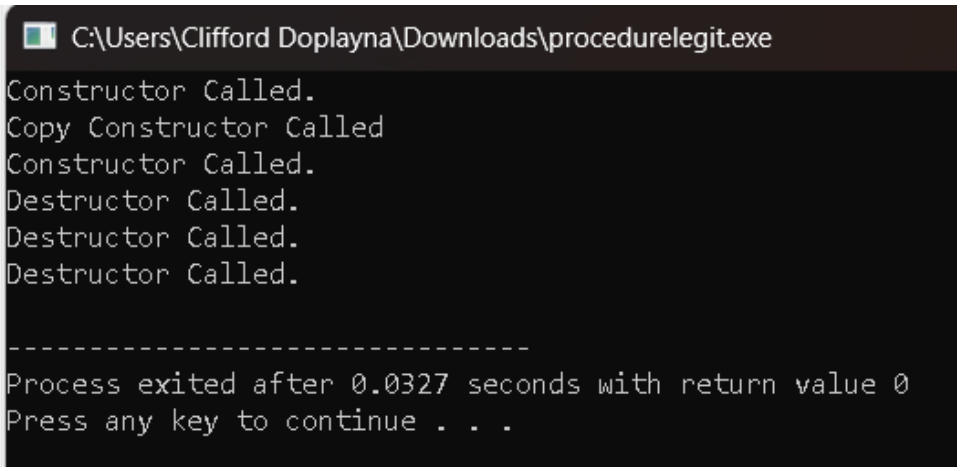
	
Observation	The console output shows three Student objects being created via the parameterized, copy, and default constructors, followed by three destructor calls when they go out of scope. The middle assignment statement prints nothing because the default assignment operator is not overloaded with a print statement.

Table 2-2. Modified Driver Program with Student Lists

Screenshot	

```
#include <iostream>
#include <string.h>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    //constructor
    Student(std::string newName = "John Doe", int newAge = 18) {
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    }

//deconstructor
~Student(){
std::cout << "Destructor Called." << std::endl;
}
//Copy Constructor
Student(const Student &copyStudent){
std::cout << "Copy Constructor Called" << std::endl;
studentName = copyStudent.studentName;
studentAge = copyStudent.studentAge;
}
//Display Attributes
void printDetails(){
std::cout << this->studentName << " " << this->studentAge << std::endl;
}
};
```

```
33  int main() {
34  const size_t j = 5;
35  Student studentList[j] = {};
36  std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
37  int ageList[j] = {15, 16, 18, 19, 16};
38
39  return 0;
40  }
```

```
C:\Users\Clifford Doplayna\Downloads\procedurelegit.exe
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.

--------------------------------
Process exited after 0.2231 seconds with return value 0
Press any key to continue . . .
```

Observation	The declaration of the studentList array triggers the default constructor five times to initialize its elements, while the parallel name and age arrays remain entirely unused. When the main function exits, the destructor is automatically called five times to clean up each constructed object in the array.

Table 2-3. Final Driver Program

Loop A	`for(int i = 0; i < j; i++){ //Loop A`
	`Student *ptr = new Student(namesList[i], ageList[i]);`
	`studentList[i] = *ptr;`
	`}`

Observation	The loop iterates j times, dynamically creating new Student objects on the heap using names and ages from the respective lists. These objects are then assigned to the studentList array by dereferencing the pointer, but because delete is never called on ptr, this loop introduces a memory leak for each dynamically allocated object.

Loop B	`for(int i = 0; i < j; i++){ //Loop B`
	`studentList[i].printDetails();`
	`}`

Observation	This loop iterates j times, calling the printDetails() member function on each element in the studentList array. This sequentially prints the assigned name and age of each student object to the console.
Output	

```cpp
#include <iostream>
#include <string.h>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    //constructor
    Student(std::string newName = "John Doe", int newAge = 18) {
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    }

//deconstructor
~Student(){
std::cout << "Destructor Called." << std::endl;
}
//Copy Constructor
Student(const Student &copyStudent){
std::cout << "Copy Constructor Called" << std::endl;
studentName = copyStudent.studentName;
studentAge = copyStudent.studentAge;
}
//Display Attributes
void printDetails(){
std::cout << this->studentName << " " << this->studentAge << std::endl;
}
};

int main() {
const size_t j = 5;
Student studentList[j] = {};
std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
int ageList[j] = {15, 16, 18, 19, 16};
for(int i = 0; i < j; i++){ //loop A
Student *ptr = new Student(namesList[i], ageList[i]);
studentList[i] = *ptr;
}
for(int i = 0; i < j; i++){ //loop B
studentList[i].printDetails();
}
return 0;
}
```

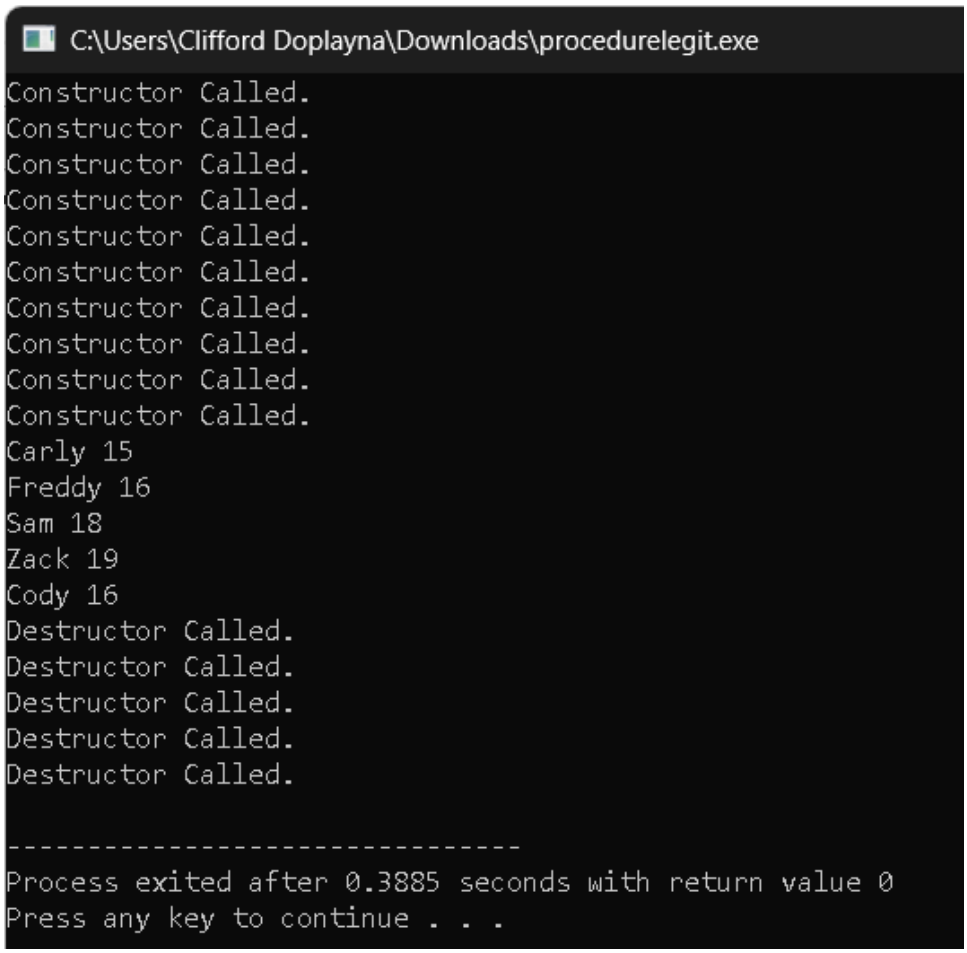
	
Observation	The output shows ten constructor from initializing the static array and five from the dynamic heap allocations in Loop A. Loop B prints the copied student details, and upon program exit, only the five static array objects are destroyed, leaving the dynamically allocated heap objects leaked in memory.

Table 2-4. Modifications/Corrections Necessary

Modifications	
Observation	

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve programming problems using dynamic memory allocation, arrays and pointers

```cpp
#include <iostream>
#include <string>
using namespace std;

class Fruit {
private:
    string name;
    double price;
    int quantity;

public:
    // Constructor
    Fruit(string n = "", double p = 0, int q = 0) {
        name = n;
        price = p;
        quantity = q;
    }

    // Destructor
    ~Fruit() {
        cout << "Fruit Destructor: " << name << endl;
    }

    // Copy Constructor
    Fruit(const Fruit& other) {
        name = other.name;
        price = other.price;
        quantity = other.quantity;
    }

    // Copy Assignment Operator
    Fruit& operator=(const Fruit& other) {
```

```
33          if (this != &other) {
34              name = other.name;
35              price = other.price;
36              quantity = other.quantity;
37          }
38          return *this;
39      }
40
41      double calculateSum() const {
42          return price * quantity;
43      }
44
45      void display() const {
46          cout << "Fruit: " << name
47               << " | Price: PHP " << price
48               << " | Quantity: " << quantity
49               << " | Total: PHP " << calculateSum()
50               << endl;
51      }
52  };
53
54  class Vegetable {
55  private:
56      string name;
57      double price;
58      int quantity;
59
60  public:
61      // Constructor
62      Vegetable(string n = "", double p = 0, int q = 0) {
63          name = n;
64          price = p;
```

```
65          quantity = q;
66      }
67
68      // Destructor
69      ~Vegetable() {
70          cout << "Vegetable Destructor: " << name << endl;
71      }
72
73      // Copy Constructor
74      Vegetable(const Vegetable& other) {
75          name = other.name;
76          price = other.price;
77          quantity = other.quantity;
78      }
79
80      // Copy Assignment Operator
81      Vegetable& operator=(const Vegetable& other) {
82          if (this != &other) {
83              name = other.name;
84              price = other.price;
85              quantity = other.quantity;
86          }
87          return *this;
88      }
89
90      double calculateSum() const {
91          return price * quantity;
92      }
93
94      void display() const {
95          cout << "Vegetable: " << name
96               << " | Price: PHP " << price
```

```
97                  << " | Quantity: " << quantity
98                  << " | Total: PHP " << calculateSum()
99                  << endl;
100     }
101 };
102
103 double TotalSum(Fruit* fruits[], int fruitCount,
104                 Vegetable* vegetables[], int vegCount)
105
106     double total = 0;
107
108     for (int i = 0; i < fruitCount; i++) {
109         if (fruits[i] != NULL)
110             total += fruits[i]->calculateSum();
111     }
112
113     for (int i = 0; i < vegCount; i++) {
114         if (vegetables[i] != NULL)
115             total += vegetables[i]->calculateSum();
116     }
117
118     return total;
119 }
120
121 int main() {
122
123     Fruit* GroceryFruits[2];
124     Vegetable* GroceryVegetables[2];
125
126     GroceryFruits[0] = new Fruit("Apple", 10, 7);
127     GroceryFruits[1] = new Fruit("Banana", 10, 8);
128
```

```
    GroceryVegetables[0] = new Vegetable("Broccoli", 60, 12);
    GroceryVegetables[1] = new Vegetable("Lettuce", 50, 10);

    cout << "===== Jenna's Grocery List =====\n\n";

    for (int i = 0; i < 2; i++)
        GroceryFruits[i]->display();

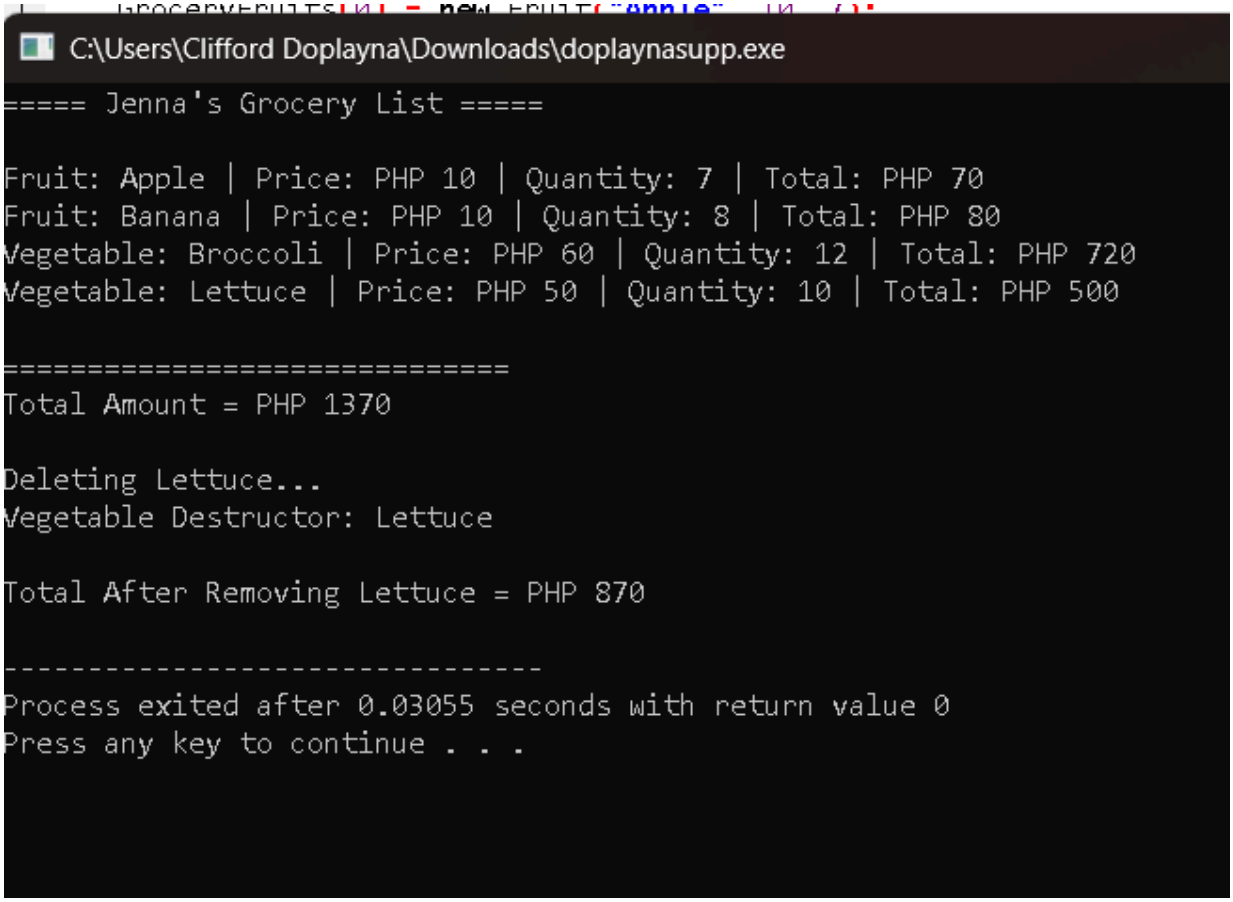
    for (int i = 0; i < 2; i++)
        GroceryVegetables[i]->display();

    cout << "\n==============================\n";
    cout << "Total Amount = PHP "
         << TotalSum(GroceryFruits, 2, GroceryVegetables, 2)
         << endl;

    cout << "\nDeleting Lettuce...\n";
    delete GroceryVegetables[1];
    GroceryVegetables[1] = NULL;

    cout << "\nTotal After Removing Lettuce = PHP "
         << TotalSum(GroceryFruits, 2, GroceryVegetables, 2)
         << endl;

    return 0;
}
```



3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chatgpt,quillbot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts,grammar checker**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used the chatgpt to search for all the informations that im confused and needed more explanations and I used quillbot for grammar checker in the conclusion.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

Conclusion

After understanding and finishing this activity, what I've learned throughout the summary of lessons was how I can implement static and dynamic allocation and also how I can create dynamic objects using pointers and arrays for my program. And lastly, was how I could solve programming problems using the lessons I learned for this activity. After finishing the procedure activity, I learned how static and dynamic memory allocation works and how it is used in different types of programs. I also learned how the constructors are used in a code and what they do to the program. For the supplementary activity it consists of all of the topics explained in the discussion and the procedure and all I needed to do was to apply all of it to finish the supplementary activity. The things that I need to improve for this activity are that I need to learn and understand it better for me to not rely too much on AI for information that is needed for the activity.

Assessment Rubric

T.I.P. S.O. 7 Rubric for Lifelong Learning

Criteria	Ratings						Pts
Acquire and apply new knowledge from outside sources	**6 pts** **Excellent** Educational interests and pursuits exist and flourish outside classroom requirements, knowledge and/or experiences are pursued independently and applies knowledge learned into practice	**5 pts** **Good** Educational interests and pursuits exist and flourish outside classroom requirements, knowledge and/or experiences are pursued independently	**4 pts** **Satisfactory** Look beyond classroom requirements, showing interest in pursuing knowledge independently	**3 pts** **Unsatisfactory** Begins to look beyond classroom requirements, showing interest in pursuing knowledge independently	**2 pts** **Poor** Relies on classroom instruction only	**1 pts** **Very Poor** No initiative or interest in acquiring new knowledge	6 pts
Learn independently	**6 pts** **Excellent** Completes an assigned task independently and practices continuous improvement	**5 pts** **Good** Completes an assigned task without supervision or guidance	**4 pts** **Satisfactory** Requires minimal guidance to complete an assigned task	**3 pts** **Unsatisfactory** Requires detailed or step-by-step instructions to complete a task	**2 pts** **Poor** Shows little interest to complete a task independently	**1 pts** **Very Poor** No interest to complete a task independently	6 pts
Critical thinking in the broadest context of technological change	**6 pts** **Excellent** Synthesizes and integrates information from a variety of sources; formulates a clear and precise perspective; draws appropriate conclusions	**5 pts** **Good** Evaluate information from a variety of sources; formulates a clear and precise perspective	**4 pts** **Satisfactory** Analyze information from a variety of sources; formulates a clear and precise perspective.	**3 pts** **Unsatisfactory** Apply the gathered information to formulate the problem	**2 pts** **Poor** Gather and summarized the information from a variety of sources but failed to formulate the problem	**1 pts** **Very Poor** Gather information from a variety of sources	6 pts
Creativity and adaptability to new and emerging technologies	**6 pts** **Excellent** Ideas are combined in original and creative ways in line with the new and emerging technology trends to solve a problem or address an issue.	**5 pts** **Good** Ideas are creative and adapt the new knowledge to solve a problem or address an issue	**4 pts** **Satisfactory** Ideas are creative in solving a problem or address an issue	**3 pts** **Unsatisfactory** Shows some creative ways to solve the problem	**2 pts** **Poor** Shows initiative and attempt to develop creative ideas to solve the problem	**1 pts** **Very Poor** Ideas are copied or restated from the sources consulted	6 pts
							Total Points: 24